Phase 1 Θοδωρής Μαλέας

1. Για να είναι τα δεδομένα σε κατάλληλη μορφή για είσοδο σε transformers τα παίρνω από το csv και τα περνάω σε lists (id και text).
2. Επιλέγω το μοντέλο all-MiniLM-L6-v2 καθώς σύμφωνα με το https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 αναφέρεται ότι «**Intended uses**
   Our model is intended to be used as a sentence and short paragraph encoder. Given an input text, it outputs a vector which captures the semantic information. The sentence vector may be used for information retrieval, clustering or sentence similarity tasks.» οπότε είναι κατάλληλο για την συγκεκριμένη περίπτωση αφού τα δεδομένα είναι σε μικρές παραγράφους. Επίσης αναφέρεται ότι «By default, input text longer than 256 word pieces is truncated.» οπότε τα κείμενα που αποτελούνται από περισσότερα tokens από 256 θα γίνει αποκοπή από το μοντέλο σύμφωνα με τη default λειτουργία του. Στη συνέχεια αφού δημιουργήσει το μοντέλο τα embeddings για τα data και τα queries αντίστοιχα αποθηκεύω τις λίστες με τα embeddings σε files (data_embeddings.pkl, query_embeddings.pkl) με το pickle.dump (γράφοντας στα files με binary mode).
3. Σύμφωνα με το https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 το μοντέλο εκπαιδεύτηκε με cosine similarity (το οποίο δίνει ομοιότητα σε δύο διανύσματα με βάση τη μεταξύ τους γωνία ανεξάρτητα του μήκους τους) οπότε θα χρησιμοποιήσω αυτό το μέτρο για να κάνω evaluate πόσο μοιάζουν δύο διανύσματα μεταξύ τους. Αν τα διανύσματα (a, b) είναι κανονικοποιημένα τότε το εσωτερικό γινόμενό τους είναι το ίδιο με το cosine similarity (αφού ||a|| = ||b|| = 1 και cos_sim(a,b)=a*b/(||a||*||b||) = a*b = dot_prod(a,b)). Οπότε θα χρησιμοποιήσω το indexFlatIP της FAISS σε κανονικοποιημενα διανύσματα με τα embeddings ώστε να έχω μία μετρική για την ομοιότητα του νοήματος μεταξύ των δύο διανυσμάτων (https://github.com/facebookresearch/faiss/wiki/FAQ/7f59d7b610e258df8c15dc808ca54b2325515375).

4. Για κάθε ερώτημα εκτελώ την εντολή της FAISS index.search() στην οποία δίνω τα query embeddings (τα query_embeddings τα έχω είδη δημιουργήσει μαζί με τα embeddings των δεδομένων IR2025) και το k (πόσους κοντινότερους γείτονες θέλω να επιστρέψει (20,30,50)). Τα αποτελέσματα τα επιστρέφει ταξινομημένα και τα αποθηκεύω στο αντίστοιχο αρχείο για κάθε k.



5. Για το mean average precision τρέχω την εντολή .\trec_eval -m map <loc: q_rels.txt> <loc: results k{k}.txt> για κάθε k (20,30,50).

k	20	30	50
Map	0.3129	0.3413	0.3744



Για το precision τρέχω την εντολή .\trec_eval -m P.{precision@k} <loc: q_rels.txt> <loc: results k{k}.txt> για κάθε k (20,30,50) και κάθε precision@k (5,10,15,20)

k	5	10	15	20
20	0.6600	0.5000	0.3933	0.3500
30	0.6600	0.5000	0.3933	0.3500
50	0.6600	0.5000	0.3933	0.3500



6. Συγκριτικά με τα αποτελέσματα της πρώτης φάσης τα αποτελέσματα που πήρα τώρα είναι χειρότερα και στο map και στο precision σε κάθε τιμή. Για να δουλεύει τόσο καλά η κλασσική ανάκτηση λογικά τα queries με τα data έχουν πολλές κοινές λέξεις ώστε να αρκεί η κλασσική ανάκτηση αν και δεν παίρνει υπ' όψη την σημασιολογία των λέξεων και ακόμα να κάνει και λιγότερα λάθη έτσι.

Άλλες πηγές:

- https://medium.com/@devbytes/similarity-search-with-faiss-a-practical-guide-to-efficient-indexing-and-retrieval-e99dd0e55e8c
- https://sbert.net/docs/package_reference/sentence_transformer/SentenceTransformer.html
- https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- https://github.com/facebookresearch/faiss/wiki